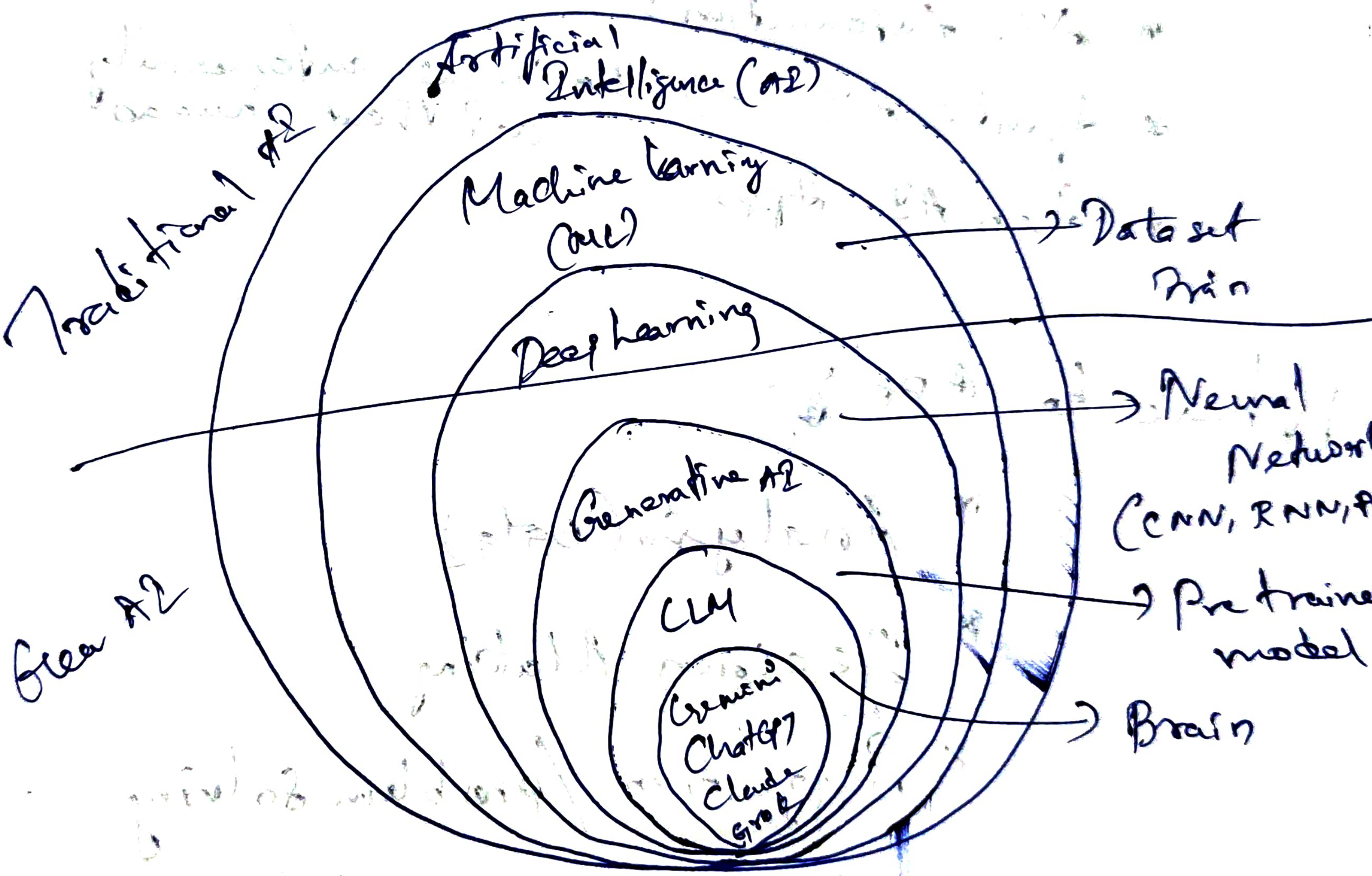
24/08/26

What is AI?

* Intelligent System
* Can do complex tasks

AI layers:



AGI - Artificial General Intelligence

ASI → Artificial Super Intelligence.

Gen AI

* Text, images, audio, video generation
  - 95% accuracy, 85% acc., 80% acc, 60% acc.

* LLM
* AI Agent
* AI, Automation :-
* Agentic AI → Do tasks autonomously without human
* Gen AI Apps

Where to Apply:

* Analysis (Data)
* Decision Making
* Suggestion / problem solving
* Generate the output.

* Domain Expert + Gen AI

Domain + Gen AI.

* Five Skills AI:

→ Gen AI application

→ AI Automation → ex: Daily status email

→ AI Agent → Workflow → Complete Reason

→ Agent AI → Goal Oriented

→ Microsaas tool →

Opportunity:

* Freelancer
* Career.
* Business Owner

25/08/26

Introduction to Python

Programming:

→ print (" Hello! This is my first Program")

Python & Uses:

→ Created by Guido van Rossum

→ Easy to read

→ Simple to learn

→ Powerful.

Use Cases:

→ Web Development

→ Data Science

→ AI

→ ML

→ Automation

→ Game dev

→ App Dev

```
marks = [80, 90, 70, 100]
average = sum(marks) / len(marks)
print ("Average marks:", average)
```

.py → file type

.ipynb → learn to know

Code

markdown

Python Syntax:

Rule 1: print()

```
print ("Welcome to Pytho")
print ("Hi.")
```

Rule 2: Comments #

```
# print the string/text given
print (" Helo!")
```

Rule 3: Indentation

```
if 5 > 2 :
    print ("5 is greater than 2")
```

Rule 4: Case Sensitive.

Python Variables:

```
name = 23
print (name)
```

```
name = "Ravi"
Name = "Kumar"  # Different Variable
```

Python Enviornment:

* A virtual Environment is a separate private space for each python project
* It keeps libraries of one project away from another project
* Version Control: version of one project doesnot affect with another project

> python -m venv myenv

> myenv/Scripts/Activate

> pip install requests

To deactivate => deactivate

pip freeze >requirements.txt

.ipynb -> To run

-> Select kernel -> our env

-> Play button

Python Operators :

+ - addition

- Subtraction

* Multiplication

/ Division

% Modulus

// floor Division

** power

Python Datatypes:

* int
* float
* Str
* bool

type () → identify the data type

Converting the data type:

x = "100"

y = int (x)

print(y+50)   ~ 150

Storing Methods: Collections

→ List [ ] ordered, mutable, Duplicate
→ tuple ( ) immutable
→ Set { } No mutable No
→ Dictionary {key:value} Yes mutable

List [ ]

→ append → add to list

Taple ( )

→ Cannot change or add value, to it

Set { }

Dict: { key : value }

Python Controlflow:

if elif else

Nestif Statements:

```
x = 100
if jo == 100:
    if shop == open:
        print("closed")
    else:
        print("open")
else:
    print("go away")
```

Python Loops:

for → while

for:

→ range( ).

```
for i in range(5):
    print("Number", i)
```

→ list [ ]

```
fruits = ["Apple", "Banana"]

for i in fruits:
    print(i)
```

while:

break & Continue:

```
for i in range(1, 6):
    if i == 3
        break;
```

Python functions:

__init__.py → module file

↓

Consider as a package in that folder

Error handling

- try
- except → catch error try

↓

except ValueError:

try:

except :

else:

finally: → this always run

except Exception as e:

print ("" {e})

Python File handling:

* w → write or overwrite
* r → read a file
* a → append to end of file.

```
with open("myfile.txt","a") as file:
    file.write("——")

with open("myfile.txt","r"):
    content = file.read()
```